

Tailwind CSS

Utility-First Styling

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Understand utility-first CSS and why it scales
- ▶ Install Tailwind v4 in a Vite project
- ▶ Style components with spacing, color, typography, and layout utilities
- ▶ Use state variants (hover, focus, disabled, dark)
- ▶ Build a responsive, dark-mode-ready UI

Section Items

- ▶ Setup
- ▶ Styling with Utilities
- ▶ Dark Mode

Where Are We?

- ▶ Just finished React fundamentals (snapshots 01–12)
- ▶ We have a working Todo app — it's **unstyled**
- ▶ Today: make it look like a real product, using Tailwind

Why Tailwind?

- ▶ Hand-rolled CSS scales badly — `.todo-card`, `.todo-card-done`, `.todo-card-hover-delete-button`
- ▶ Naming things is hard. Keeping selectors in sync with components is harder.
- ▶ **Tailwind:** skip the names. Compose styles from utility classes right on the element.

Old Way vs Tailwind

```
/* CSS file */
.todo-card {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  padding: 0.5rem;
}
.todo-card:hover { background: #f9fafb; }
```

```
{/* Tailwind */}
<li className="flex items-center gap-3 p-2 hover:bg-gray-50">
```

Same result. No CSS file. No naming.

What People Worry About

- ▶ “My className is going to be huge!” — Yes. It’s fine. Components encapsulate it.
- ▶ “Won’t the bundle be huge?” — No. Tailwind ships only the classes you use.
- ▶ “How do I remember all the classes?” — VS Code’s Tailwind IntelliSense autocompletes everything.

Section Items

► Setup

- Styling with Utilities
- Dark Mode

Tailwind v4 Install (Vite)

```
npm install tailwindcss @tailwindcss/vite
```

vite.config.ts:

```
import tailwindcss from "@tailwindcss/vite";  
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
});
```

src/index.css:

```
@import "tailwindcss";
```

That's it. No tailwind.config.js. No PostCSS config.

Proof It Works

- ▶ Add `className="text-4xl font-bold text-blue-600"` to the H1
- ▶ Save → browser shows huge blue heading
- ▶ Everything else still unstyled

Install the IntelliSense Extension

- ▶ VS Code → Extensions → “Tailwind CSS IntelliSense”
- ▶ Type `bg-` and get a color picker dropdown
- ▶ Without it, you're guessing

Section Items

- Setup
- ▶ **Styling with Utilities**
- Dark Mode

The 4 Utility Categories

1. **Spacing** — `p-`, `m-`, `gap-`, `space-x-`
2. **Color & typography** — `bg-`, `text-`, `font-`
3. **Layout** — `flex`, `grid`, `items-`, `justify-`
4. **State variants** — `hover:`, `focus:`, `disabled:`, `dark:`

Spacing and Layout

```
<div className="min-h-screen bg-gray-50 py-8 px-4 sm:py-12">  
  <div className="max-w-md mx-auto bg-white rounded-2xl  
    shadow-md p-6 sm:p-8">
```

- ▶ `min-h-screen` — fill viewport
- ▶ `max-w-md mx-auto` — center card, cap at ~28rem
- ▶ `rounded-2xl shadow-md` — card look
- ▶ `p-6 sm:p-8` — padding bumps up on larger screens

The Spacing Scale

Class	Value
p-0	0px
p-1	0.25rem (4px)
p-2	0.5rem (8px)
p-4	1rem (16px) ← default rhythm
p-8	2rem (32px)

Same scale for m-, gap-, space-x-, etc. Learn ONE scale, it works everywhere.

Typography

```
<p className="text-sm text-gray-600 mb-4">
```

- ▶ Size: text-xs, text-sm, text-base, text-lg, text-xl, text-2xl, ...
- ▶ Weight: font-normal, font-medium, font-semibold, font-bold
- ▶ Color: text-gray-600 — gray scale runs 50 (lightest) → 950 (darkest)

Color Naming

- ▶ `text-` for text, `bg-` for background, `border-` for borders, `accent-` for checkboxes
- ▶ Every color has a -50 through -950 scale
- ▶ Pick a family, stick to it (we use `gray-` + `blue-` + `red-`)

Flexbox

```
<div className="flex gap-2 mb-4">
```

- ▶ `flex` — turn on flexbox (default direction: row)
- ▶ `gap-2` — 0.5rem between children
- ▶ `items-center` — vertical align center
- ▶ `justify-between` — push to opposite edges
- ▶ `flex-col` — switch direction to column

State Variants

```
<button className="bg-gray-100 hover:bg-gray-200  
            disabled:bg-gray-300">
```

- ▶ `hover:` — on mouse hover
- ▶ `focus:` — on keyboard focus
- ▶ `disabled:` — when `disabled` prop is true
- ▶ All chain together: `hover:focus:bg-blue-500`

Live Code: State Variants

- ▶ Hover the filter buttons → color changes
- ▶ Click one → it goes blue and disabled

Dynamic Classes

```
className={
  "px-3 py-1 rounded-full " +
  (isActive
    ? "bg-blue-600 text-white"
    : "bg-gray-100 hover:bg-gray-200")
}
```

- ▶ Just a string — concatenate conditionally
- ▶ This is the everyday pattern in React + Tailwind
- ▶ Helper libraries like `clsx` make this prettier — covered later

Group Hover

```
<li className="group flex items-center gap-3
            hover:bg-gray-50">
  ...
  <button className="opacity-0 group-hover:opacity-100">
    delete
  </button>
</li>
```

- ▶ group on parent → group-hover: on child
- ▶ The delete button only appears when you hover its row

Live Code: Group Hover

- ▶ Hover a row → delete button fades in

Forms

```
<input className="px-3 py-2 rounded-lg border border-gray-300  
  focus:outline-none focus:ring-2 focus:ring-blue-500" />
```

```
<button disabled={isEmpty}  
  className="bg-blue-600 hover:bg-blue-700  
    disabled:bg-gray-300 disabled:cursor-not-allowed">
```


- ▶ Tab through inputs → focus ring jumps
- ▶ Empty input → Add button greyed out

Responsive

```
<div className="p-6 sm:p-8">  
<div className="py-8 sm:py-12">
```

- ▶ sm: = 640px+, md: = 768px+, lg: = 1024px+
- ▶ Tailwind is **mobile-first**: base = mobile, sm: overrides at 640px+

Other Useful Utilities

- ▶ `divide-y divide-gray-100` — automatic borders between list children
- ▶ `transition-colors` — animate color changes smoothly
- ▶ `tabular-nums` — fixed-width digits (good for minute counts)
- ▶ `cursor-pointer`, `cursor-not-allowed`
- ▶ `line-through` for done todos

Section Items

- Setup
- Styling with Utilities
- ▶ **Dark Mode**

dark: Is Just Another Variant

```
<div className="bg-gray-50 dark:bg-gray-900">  
<p className="text-gray-800 dark:text-gray-100">
```

- ▶ Same syntax as `hover:` and `focus:`
- ▶ Tailwind v4 default: triggers on `prefers-color-scheme: dark`

Class-Based Dark Mode

In `index.css`:

```
@custom-variant dark (&:where(.dark, .dark *));
```

- ▶ “The `dark` variant fires when there’s a `.dark` class anywhere up the ancestor chain”
- ▶ Lets you build a user-controlled toggle

The Toggle Hook

```
export function useDarkMode() {  
  const [isDark, setIsDark] = useState(/* ... */);  
  useEffect(() => {  
    document.documentElement.classList.toggle("dark", isDark);  
    localStorage.setItem("theme", String(isDark));  
  }, [isDark]);  
  return { isDark, toggle: () => setIsDark((p) => !p) };  
}
```

- ▶ Same custom-hook pattern as useTodos
- ▶ Toggles `.dark` on `<html>`, persists to `localStorage`

The Work

- ▶ Every color class gets a dark counterpart
- ▶ `bg-white` → `bg-white dark:bg-gray-800`
- ▶ `text-gray-800` → `text-gray-800 dark:text-gray-100`
- ▶ Tedious but mechanical — AI can do this for you

Live Code: Dark Mode

- ▶ Click toggle → instant flip
- ▶ Refresh → preference persists

Where This Leads

- ▶ Component libraries built on Tailwind: shadcn/ui, daisyUI, Headless UI
- ▶ `cn()` helper (`clsx` + `tailwind-merge`) for cleaner conditional classes
- ▶ Custom design tokens with `@theme` directive in CSS

What You Can Now Do

- ▶ Style any element with utility classes ✓
- ▶ Build a responsive layout with `flex + sm:/md:` ✓
- ▶ Add hover/focus/disabled states ✓
- ▶ Toggle dark mode ✓

Before Next Class

- ▶ Pick one of your portfolio pages, restyle it in Tailwind
- ▶ No CSS file — classes only